

Lab0- 一些Python语言的入门

顾运-2026秋

联系方式:

yungu@ieee.org

Python-101: 关于开发平台&虚拟环境配置

如果你有丰富的Python编程经验，这次作业可以跳过。

如果想要光速复习一下Python基本语法，请参考Canvas上的一小时速通Python视频，并快速完成本次作业。

注意，本课程对于你的操作系统，开发环境都没有具体要求，Windows/Linux/Mac都可以，PyCharm/VSCode/记事本都可以

Python包管理

Python目前有两种比较好的包管理手段，一种是基于conda，另一种是基于uv。前者上手容易更无脑，后者更为现代化并且管理效率高。我们这里简要展示一下如何基于conda来创建虚拟环境。

可以首先基于Anaconda[下载安装包到本地](#)，执行安装。如果你使用的是macos或者linux，打开你的bash；如果你使用的是windows，可以在你的开始菜单中（或者搜索中）找到Anaconda Prompt。此时会加载到Anaconda的Base环境中，例如下图

```
(base) yungu@yundeMacBook-Air ~ %
```

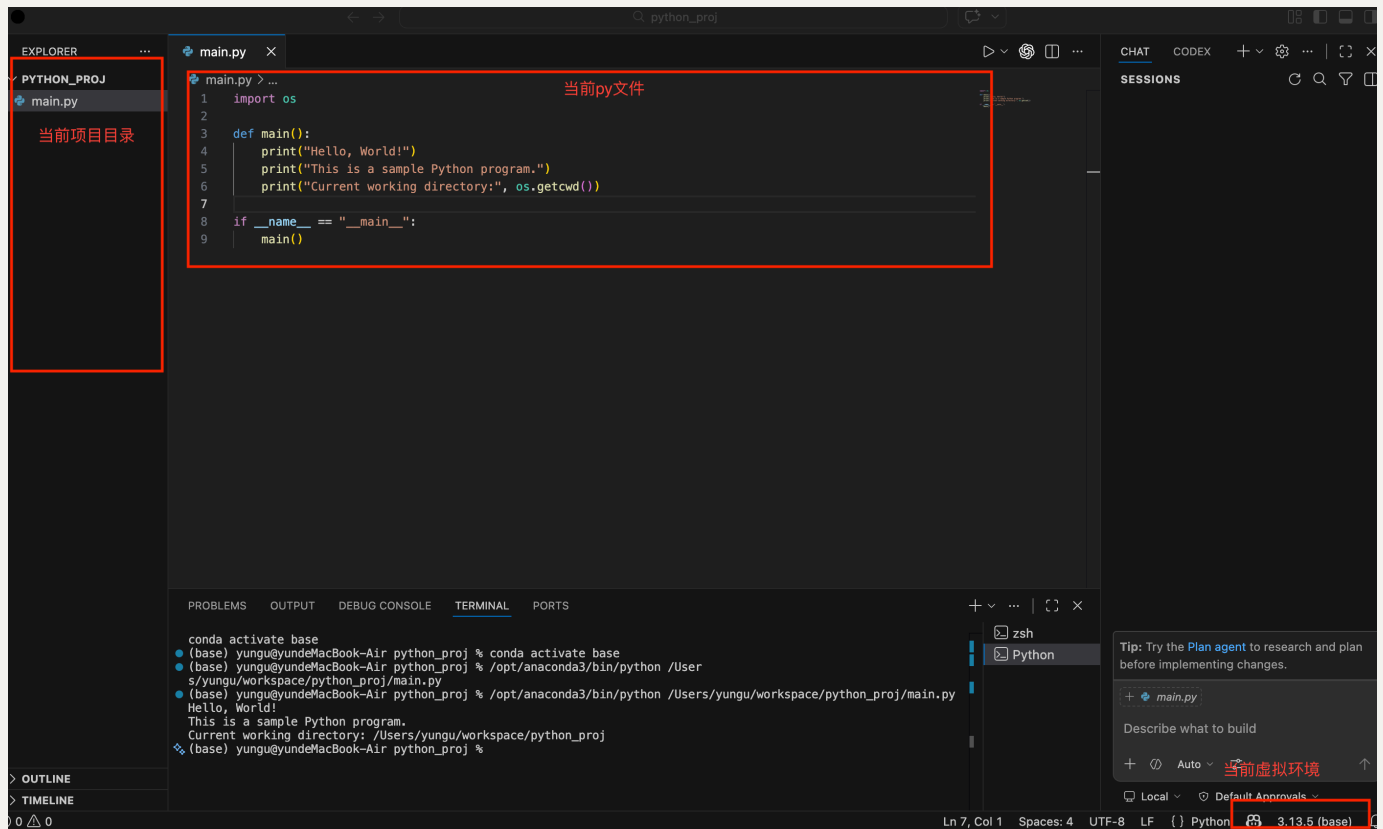
一般建议每个project独立建立一个虚拟环境（venv），如何建立虚拟环境请参考我的速通视频。由于Python涉及到大量的package管理，比如我们课程涉及到的pygame，在Anaconda中是没有默认安装的。那么如何安装package呢？你可以直接在bash里面使用conda或者pip安装相应的包，比如采用 `pip install pygame`：

```
[(base) yungu@yundeMacBook-Air ~ % conda activate au3323
(au3323) yungu@yundeMacBook-Air ~ % pip install pygame
Collecting pygame
  Downloading pygame-2.6.1-cp313-cp313-macosx_11_0_arm64.whl.metadata (12 kB)
  Downloading pygame-2.6.1-cp313-cp313-macosx_11_0_arm64.whl (12.4 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.4/12.4 MB 9.5 MB/s 0:00:01
Installing collected packages: pygame
Successfully installed pygame-2.6.1
(au3323) yungu@yundeMacBook-Air ~ %
```

此时，我们通过pip安装了一个叫做pygame的包。

面向Python的开发环境

现有的开发环境选择非常多，你可以使用PyCharm（JetBrains全家桶之Python专款），VSCode（目前最主流的开发环境）或者Cursor（基于VSCode的改版，AI辅助编程便捷）。我们简要介绍基于VSCode的开发环境搭建。打开你的VSCode并打开当前Python项目所在的目录，如下图：



你可以点击右下角的虚拟环境，将当前项目切换至你自己创建的虚拟环境。想要运行代码，你可以点击右上角的“Run”按钮，也可也在TERMINAL中直接输入命令执行，比如上图中的"python /Users/yungu/workspace/python_proj/main.py"。

Python-101: 语言快速熟悉

本次作业用于熟悉后续人工智能课程中需要使用的 Python 基础。本次作业不计分，也无需提交。

本次作业以 **coding** 为主。完成后，你应该能够：

- 使用变量、条件判断与循环；
- 使用 `list`、`tuple`、`set` 和 `dict`；
- 编写并调用函数；
- 创建和使用 Python `class`；
- 理解 `__init__`、`self`、属性和方法；
- 使用 NumPy 完成基本向量运算；
- 使用 Python 表示 AI 中常见的 `state`、`action` 和 `environment`；

- 阅读、补全并调试一个简单的 Python 程序。

建议环境：

```
Python >= 3.10  
NumPy
```

安装 NumPy：你可以在terminal环境内使用pip，或者采用IDE内的安装包工具。如果在terminal内，请使用

```
pip install numpy
```

运行与处理、

运行方式：

```
python homework.py
```

请不要修改已经给出的：

- 函数名；
- 类名；
- 参数名；
- 测试代码的整体结构。

所有需要完成的位置均使用：

```
# TODO
```

提供了标记。

Problem 1. Basic Python Programming

完成函数：

```
def basic_statistics(numbers):  
    """  
    Args:  
        numbers: a non-empty list of numbers  
  
    Returns:  
        total, mean, maximum, minimum  
    """
```

例如：

```
numbers = [3, 7, 2, 9, 4]  
print(basic_statistics(numbers))
```

期望输出：

```
(25, 5.0, 9, 2)
```

要求：本题不要使用 `sum()`、`max()`、`min()`。

Problem 2. Conditional Statements

完成：

```
def get_grade(score):
```

评分规则：

SCORE	GRADE
90–100	A
80–89	B
70–79	C
60–69	D
< 60	F

若 `score < 0` 或 `score > 100`，返回：

```
Invalid
```

Problem 3. Lists and Loops

完成：

```
def filter_numbers(numbers, threshold):
```

返回所有严格大于 `threshold` 的数字。

例如：

```
filter_numbers([3, 8, 1, 9, 4, 7, 2, 10], 5)
```

返回：

```
[8, 9, 7, 10]
```

然后完成：

```
def count_even_odd(numbers):
```

返回：

```
(number_of_even_numbers, number_of_odd_numbers)
```

Problem 4. Dictionary

给定：

```
values = {  
    "A": 3.0,  
    "B": 7.5,  
    "C": 2.5,  
    "D": 6.0  
}
```

完成：

```
def best_state(values):
```

返回 value 最大的 state。

本题不要使用：

```
max(values, key=values.get)
```

继续完成：

```
def update_value(values, state, new_value):
```

若 state 已存在，则更新它；若不存在，则创建新条目。

Problem 5. Functions and Distance

二维位置用 tuple 表示：

```
p = (x, y)
```

5.1 Manhattan Distance

完成：

```
def manhattan_distance(p1, p2):
```

定义：

$$d = |x_1 - x_2| + |y_1 - y_2| \quad (1)$$

例如：

```
manhattan_distance((2, 3), (7, 5))
```

返回：

```
7
```

5.2 Euclidean Distance

完成：

```
def euclidean_distance(p1, p2):
```

定义:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad (2)$$

例如:

```
euclidean_distance((0, 0), (3, 4))
```

返回:

```
5.0
```

Problem 6. Introduction to Class

完成 `Point` 类:

```
class Point:

    def __init__(self, x, y):
        ...

    def move(self, dx, dy):
        ...

    def distance_to(self, other):
        ...

    def get_position(self):
        ...
```

示例:

```
p1 = Point(1, 2)
p2 = Point(4, 6)

print(p1.get_position())
p1.move(2, 1)
print(p1.get_position())
print(p1.distance_to(p2))
```

输出约为:

```
(1, 2)
(3, 3)
3.1623
```

本题重点:

- `class`
- `__init__`
- `self`
- object attributes
- object methods

Problem 7. Build an Agent Class

完成 `Agent` 类。

`Agent` 在二维网格中具有一个位置，并可以执行:

```
UP
DOWN
LEFT
RIGHT
```

完成：

```
class Agent:

    def __init__(self, x, y):
        ...

    def get_position(self):
        ...

    def set_position(self, position):
        ...

    def move(self, action):
        ...
```

约定：

```
UP      -> (x - 1, y)
DOWN    -> (x + 1, y)
LEFT    -> (x, y - 1)
RIGHT   -> (x, y + 1)
```

同时维护：

```
self.history
```

用于保存 Agent 经过的所有位置。

例如：

```
agent = Agent(0, 0)
agent.move("DOWN")
agent.move("DOWN")
agent.move("RIGHT")

print(agent.history)
```

应输出:

```
[(0, 0), (1, 0), (2, 0), (2, 1)]
```

Problem 8. Build a GridWorld Class

地图:

```
grid = [
    [0, 0, 0, 0],
    [0, 1, 1, 0],
    [0, 0, 0, 0],
    [1, 0, 0, 0]
]
```

其中:

```
0 = free space
1 = obstacle
```

完成:

```
class GridWorld:
```

```
def __init__(self, grid):  
    ...  
  
def get_height(self):  
    ...  
  
def get_width(self):  
    ...  
  
def is_inside(self, position):  
    ...  
  
def is_obstacle(self, position):  
    ...  
  
def is_valid(self, position):  
    ...
```

要求：一个合法位置必须同时满足：

1. 位于地图内部；
2. 不是障碍物。

Problem 9. State Transition

在 `GridWorld` 类中完成：

```
def get_next_state(self, state, action):
```

实现：

```
state + action -> next state
```

例如：

```
(2, 2) + UP -> (1, 2)
```

如果新位置超出地图边界，或是障碍物，则保持在原位置。

例如：

```
world.get_next_state((1, 0), "RIGHT")
```

由于 (1, 1) 是障碍物，因此返回：

```
(1, 0)
```

Problem 10. Valid Actions

在 `GridWorld` 类中完成：

```
def get_valid_actions(self, state):
```

返回所有执行后会真正移动到新位置的合法动作。

例如：

```
world.get_valid_actions((0, 0))
```

应得到：

```
["DOWN", "RIGHT"]
```

动作顺序可以不同。

Problem 11. NumPy

完成：

```
def vector_operations(a, b):
```

输入：

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])
```

返回：

```
addition  
subtraction  
elementwise_product  
dot_product
```

其中：

```
addition = [5, 7, 9]  
dot_product = 32
```

Problem 12. Mini Project — Robot in a GridWorld

给定：

```
grid = [  
    [0, 0, 0, 0, 0],  
    [0, 1, 1, 0, 0],  
    [0, 0, 0, 0, 0],  
    [0, 1, 0, 1, 0],  
    [0, 0, 0, 0, 0]  
]
```

以及：

```
agent = Agent(0, 0)
```

完成：

```
def run_agent(world, agent, actions):
```

规则：

1. 读取 Agent 当前状态；
2. 调用 `world.get_next_state()`；
3. 更新 Agent；
4. 保存 Agent 的运动历史；
5. 返回最终位置。

再完成：

```
def reached_goal(agent, goal):
```

如果 Agent 到达目标，返回 `True`，否则返回 `False`。

Bonus. Greedy Robot

完成：

```
def greedy_agent(world, start, goal, max_steps=100):
```

每一步：

1. 获取当前状态的所有合法动作；
2. 计算动作对应的 `next_state`；
3. 计算 `next_state` 到 goal 的 Manhattan distance；
4. 选择距离最小的 next state；
5. 重复直到到达目标或达到 `max_steps`。

返回路径：

```
[  
    (0, 0),  
    (1, 0),  
    ...  
]
```

为了避免机器人在局部区域无限循环，可以维护一个：

```
visited
```

集合。

后续课程中的联系

本次作业最后实现的结构：

```
Environment
  ↓
State
  ↓
Action
  ↓
Next State
```

将在后续课程中继续使用：

```
BFS
DFS
Uniform Cost Search
A*
CSP
MDP
Reinforcement Learning
```

因此，请尽量理解代码中 `state`、`action`、`next_state` 的含义，而不仅仅是得到正确输出。